

A concurrency-based game environment

Project Plan
Henry Frodsham
CS3821 – BSc Final Year Project

Supervisor: Felix Bradley
Department of Computer Science
Royal Holloway, University of London

1 Abstract

Split screen play is a permutation of local multiplayer allowing multiple players to play on a singular device, split screen play often allows up to 4 players to play on the same display, but each player requires a separate instance of the game to be run. Real time strategy games face more difficulty when implementing split screen play due to late game complexity and AI simulation causing significant performance issues. Concurrency is a required aspect of split screen games due to the shared state between different game instances, two game instances trying to access the same shared resource causes a race condition that must be prevented or resolved.

The core challenge faced by my project is that split screen requires multiple simultaneous game simulations, unlike traditional single player games where one player has access to 1 hardware set, each split screen player must compete for the same memory, CPU and GPU. This creates 3 critical problems: Race conditions, Frame Rate and Inputs. Unhandled race conditions could cause unpredictable behaviour or crashes within my project so access of shared resources between game instances must be mediated properly. the next critical challenge is a consistent frame rate will need to be achieved regardless of the number of players or stage of the game, my project must be able to handle storing and handling entities efficiently, on top of this I need to ensure each game instance is treated equally in terms of processing time. Split-screen play with multiple input methods is the next critical challenge faced by my project; each player must have a unique input method that should only be able to control their portion of the screen. A major challenge faced by my project is implementing split screen play in both an efficient and a user-friendly manor; if improperly implemented then end users of my project may find difficulty in using the UI on their portion of the screen so a properly scaled UI and suitable font size will be crucial for my project.

Solving this core challenge within my project could produce a unique experience that takes once beloved elements of early 21st century gaming and combines them with real time strategy elements; the high-performance aspect of my project also solves the common pitfalls of current real time strategy games with regards to late-game complexity and larger map sizes causing performance issues.

My project will implement a hybrid of entity component system architecture and event architecture for efficient contiguous data storage and convenient interthread communication. ECS architecture offers scalability compatible with the aims of my project as large amounts of game objects will need to be stored [1]. Event architecture is perfect for my project as it provides race condition avoidance utilising event queues to manage shared resource access between instances [2]. To be more specific, my project will store real time strategy related game objects such as cities and troops using ECS; to handle concurrency between separate players I will separate game instances into different threads and mediate resource access using event queues, in instances where controlling access through an event queue isn't possible then shared mutex locks will be used to prevent race conditions. To properly optimise split screen functionality in my project I'll be using multi-viewport rendering architecture where game instances use viewports into a centralized game world instead of rendering the entire game map separately. To ensure UI and input responsiveness I will separate the game logic and UI into separate threads for each game instance, the UI threads will be synchronized at the end of each frame however game logic will run separately to prevent UI slowdown. Since Ogre3d does not include appropriate end user UI support I will be using CEGUI for the main game UI, to prevent obscuring the small split off screens I will use an on-demand UI system including context menus. To facilitate prototype and debug builds of my project I will use Dear ImGui to easily create test UI's instead of spending a large amount of time creating an end user UI.

If time permits, then a secondary goal for my project is to produce detailed communication between players including a chat between players to discuss game strategy and game specific interactions such as trading. Another secondary goal that I'll pursue with my

project is graphics fidelity using shaders, my project will pursue an ultra-simplistic art style to prevent high time consumption but still providing visually appealing graphics.

2 Timeline

Ideally, my plan will be to focus on architectural groundwork at the start of term 1 with an emphasis on robust and efficient code whilst producing simple implementations for regular prototyping. Term 2 will focus on producing advanced implementations that keep user experience in mind.

2.1 Term 1

Week 1: write unit tests following TDD and setup build dependencies

Week 2: research efficient ECS and event architecture

Week 3: research input methods for controller and keyboard

Week 4: code central event bus and ECS registry class

Week 5: code input listeners + initial world and instance class

Week 6: code initial split screen functionality and implement event queue

Weeks 7: code entity factories and create entity components for ECS

Week 8-9: create initial game map with partial game functionality

Weeks 10-11: prepare interim transcript and improve implementation for presentation

2.2 Term 2

Week 1-2: revisit architecture and refactor if needed

Week 2-3: 3d model sourcing and view model adjustments

Week 4-5: improve game functionality and refactor UI

Week 6-9: evaluate results thus far, revisit objectives if needed/secondary objectives

Week 10-11: final report and viva

3. Risks and Mitigations

3.1 Hardware Failure

Throughout working on my project it's possible that my main hardware fails and subsequently I will lose the ability to work on the project. I will mitigate this risk by uploading my codebase and my written reports to the department's GitLab regularly. It's also possible that hardware I'm using specific to my project such as multiple games controllers fails and I'm unable to simulate the desired use case of handling split screen functionality. In this case I will use a software-based solution such as Keyboard Splitter to simulate multiple games controllers or in the case of keyboard failure I can use JoyToKey.

3.2 Poor time management / unrealistic goals

Attainable time frames are critical for the development of my project to reach my desired end goal. To ensure I tackle goals in a time efficient manner I will research dependencies required before hand and split the task into smaller goals and make informed decisions about a realistic time frame for each. It's also possible that I reach a plateau in my progress towards a goal thus extending the time frame required. To mitigate this, I will research similar open source projects implementing the same technology or technique as inspiration for implementing into my project.

3.3 Homogenous testing environment

Throughout development of my project it's possible that I overlook the potential for end users using a different hardware combination or a different input method such as a PS4 controller or using AMD as their graphics card manufacturer. It's crucial that I test my project across differing hardware such as a less

powerful CPU so I can avoid overlooking performance issues just because my hardware combination runs the project smoothly. I'll mitigate this risk using VMware to simulate less powerful hardware in addition to using containerization software such as Docker to test the project on other computers.

3.4 Overhead of shader and 3d model creation

Whilst high quality graphics isn't the focus of my project, I will need some extra knowledge to create 3d meshes and ensuring that these tasks aren't too time consuming will be critical for completing the project within the given time frame. To prevent these tasks being too time consuming I'll use Blender to create both 3d meshes and skeleton animations then use the Blender2Ogre plugin to convert into the required format, if a particular 3d mesh is very complex then I'll find a premade model on a free assets store such as BlendSwap.

3.5 Neglect of reports

It's possible that throughout development of the project I end up lagging behind on the reports aspect of the project due to getting enveloped in the coding aspect of the project. To mitigate the risk of this occurring I'll follow a life cycle of completing code required for a sub task then including it in my report before starting the next sub task.

3.6 Inefficient code

Due to the nature of my project it's critical that underlying code is written as efficiently as possible since it's likely that the code will be run concurrently across all users and one inefficient segment of code could have large implications for the performance of my project. To mitigate the risk of inefficient code slowing down the project will test the speed of execution of certain high priority functions using the chrono library.

Acronyms

ECS: entity component system

TDD: test driven development

Glossary

JoyToKey: computer software used to transform controller input into keyboard input

Blender2Ogre: a plugin for Blender to convert 3d meshes into an Ogre3d friendly format

BlendSwap: a website focused on providing free blender files for 3d models

Keyboard Splitter: a program used to simulate games controllers

GitLab: an online platform used to store and manage git repositories

Dear ImGui: a C++ graphics library primarily used for rapid deployment prototype builds

CEGUI: a C++ graphics library popular for game development

References

[1] Webster, Carson. "Entity Component Systems Usefulness."

[2] <https://gameprogrammingpatterns.com/event-queue.html>